

TRPO

LAMDA-5

June 14, 2026

我们这次的目标是实现算法 TRPO (**Trust Region Policy Optimization**, <https://arxiv.org/pdf/1502.05477>), 是 2015 年由当时在 UC Berkeley 读博的 John Schulman 及其导师 Pieter Abbeel 等人所提出, 发表在 JMLR, 目前引用量已过万。它成功地将传统运筹学中的“信赖域”方法引入深度强化学习, 首次在深度神经网络模型上为策略优化提供了“单调改进”的数学保证, 这意味着原则上每次更新都会让策略变得更好, 不会出现灾难性退步。

1 代码补全

该项目的实现参考了<https://spinningup.openai.com/en/latest/algorithms/trpo.html>, 请补全文件 `algorithm/trpo.py` 中的 **TODO** 部分 (完善函数 `compute_advantages_from_traj`, `compute_advantages_from_rollout`, 以及 `_update_policy`, 相关功能要求已在注释中标出)。并回答下述问题:

1.1 `collect_trajectories` 实现部分

1. `collect_trajectories` 与普通 rollout 采样方式相比, 数据收集的停止条件分别是什么?

回答: 普通 rollout (`interact_with_envs`) 按固定步数截断: 每个 epoch 每个并行环境交互满 `interact_per_epoch` 步即返回, 不等 episode 结束。

`collect_trajectories` 按轨迹条数截断: 外层循环条件为 `while not self.traj_rollout.full`, 直到存满 `trajnum` 条轨迹才停止; 每条轨迹以“环境终止”或“达到 `max_episode_length`”为内部结束条件。

2. 在 trajectory 采样模式下, 每一条轨迹需要保存哪些信息? 这些信息分别在后续 TRPO 更新中起什么作用?, 当环境返回终止信号时, 轨迹又会如何结束? 未达到最大长度的部分又应如何处理?

回答: 每条轨迹保存 `states`、`actions`、`rewards`、`dones`、`log_probs`, 对应 `add_traj()` 的输入字段:

- `states`: 供 actor 重算 $\log \pi_{\theta}(a|s)$, 供 critic 估计 $V(s_t)$ 。
- `actions`: 与 `states` 配合计算重要性采样比率。
- `rewards`: 从后往前递推 GAE 和 return。
- `dones`: 递推时乘以 $(1 - d_t)$ 切断 episode 边界的 bootstrap。
- `log_probs`: 旧策略概率, 计算 $\exp(\log \pi_{\theta} - \log \pi_{\theta_{old}})$ 。

环境返回终止信号时立即调用 `add_traj()` 保存当前轨迹并重置该 env 的缓冲区。未填满 `max_episode_length` 的剩余位置保持零初始化, `masks` 为 0, 后续用 `masks==1` 过滤掉 padding。

3. `TrajectoryRollout` 中的 `masks` 变量有什么作用? 如果不使用 `masks`, 计算 return 或 advantage 时可能出现什么问题?

回答: `masks[i,t]=1` 表示该位置是真实 transition, `=0` 表示 padding。 `TrajectoryRollout` 预分配形状为 `(trajnum, max_episode_length)`, 短轨迹后必有零填充, `add_traj()` 中 `self.masks[pos,:traj_len]=1` 记录真实长度。

若不使用 `masks`, padding 的零奖励和零状态会污染 return/advantage 的均值与方差, advantage normalization 随之偏移; GAE 递推也可能把无效步当成真实 transition, 导致 critic 和 actor 的梯度方向失真。

4. `collect_trajectories` 在收集完一批轨迹后为什么需要重新初始化环境状态？

回答：当缓冲区满时，某些环境可能停在 episode 中途，其 `observations` 是非起点状态。若不重置，下一轮轨迹从这些中间状态开始，破坏“从初始分布出发”的假设，轨迹边界不干净。`initialize()` 确保每轮采样从 episode 起点开始，使 on-policy 的“采样—更新—清空”流程清晰，避免上一批残留状态污染下一批新策略的数据。

1.2 `_update_policy` 实现部分

该部分理论主要依赖于 **Trust Region Policy Optimization** 中的 **Theorem 1**，代码实现参考 <https://spinningup.openai.com/en/latest/algorithms/trpo.html> 中的 **Algorithm 1**：

1. 解释该公式中每一项的实际意义，这个公式在什么情况下可以保证策略的单调改进？由此直观解释 TRPO 算法的设计思路（优化目标和约束的设计？先概述你的大致想法，具体细节在后续提问中体现）。

回答：TRPO 的 Theorem 1：

$$\eta(\pi_{\text{new}}) \geq L_{\pi_{\text{old}}}(\pi_{\text{new}}) - C D_{\text{TV}}^{\max}(\pi_{\text{old}}, \pi_{\text{new}})^2.$$

η 是新策略真实期望回报； $L_{\pi_{\text{old}}}$ 是固定旧策略状态分布下的代理目标（重要性采样加权 advantage）；后项是新旧策略 TV 距离的惩罚，量化用旧分布近似新分布的误差。只要代理目标提升量超过惩罚项，下界上升，策略性能单调改进。由此 TRPO 的设计思路是：最大化 L （鼓励 advantage 为正的動作）同时约束策略变化幅度，即在“可信域”内找最优更新。

2. 和文章 *Approximately optimal approximate reinforcement learning* 中的 Theorem 4.1（即 TRPO 论文中的公式 (6)）作对比，TRPO 作者在分析上的处理技术有什么具体的不同，使得 TRPO 可以推广到所有一般的随机策略类而不是给定的混合策略类？

回答：Kakade-Langford 要求新策略为混合形式 $\pi_{\text{new}} = (1 - \alpha)\pi_{\text{old}} + \alpha\pi'$ ，偏离旧策略的概率显式等于 α ，分析局限于此混合策略类。TRPO 令

$$\alpha = \max_s D_{\text{TV}}(\pi_{\text{old}}(\cdot|s), \pi_{\text{new}}(\cdot|s))$$

作为“单步采到不同动作的概率上界”，不要求 π_{new} 有特定结构，从而推广到任意随机策略（包括神经网络参数化的高斯策略）。

3. TRPO 中策略更新的 surrogate objective $L_{\pi_{\text{old}}}(\pi_{\text{new}})$ 一项在实际算法中是如何设计的？

回答：用旧策略采样数据做 Monte Carlo 估计，重要性采样修正采样偏差：

$$\hat{L}(\theta) = \frac{1}{N} \sum_{t=1}^N \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t.$$

对应 `trpo.py`：先保存采样时的 `old_log_probs`，更新时用 `self.agent.log_prob(states, actions)` 重算新策略概率，再令 `ratio = torch.exp(new_log_probs - old_log_probs)`，最后 `surrogate = (ratio * advantages).mean()` 即为 actor 最大化的目标。

4. 为什么 TRPO 需要约束新旧策略之间的 KL divergence 而不是定理中的 TV 距离？在实际算法中我们采用 KL 散度的二阶近似 $D_{\text{KL}}(\pi_{\theta_{\text{old}}} \parallel \pi_{\theta}) \sim \frac{1}{2} \Delta \theta^T (\nabla_{\theta}^2 D_{\text{KL}}) \Delta \theta$ ，延续该思路解释为什么在 Algorithm 1 中我们会采用 line search 搜索参数 (Algorithm 1 line 8&9)？

回答：KL 散度对高斯策略有解析式且可微，TV 距离不具备此优势；Pinsker 不等式保证 KL 控制 TV，故用 KL 替代。KL 的二阶近似 $\frac{1}{2} \Delta \theta^T H \Delta \theta \leq \delta$ (H 为 Fisher 信息矩阵) 将约束转化为线性方程 $Hx = g$ ，`trpo.py` 用共轭梯度求解得自然梯度方向，再由 `stepsize = sqrt(2*delta / xHx)` 算出理论步长。

但二阶近似在非线性网络下仅局部有效，实际 KL 可能超过 δ 而 surrogate 也可能不提升，因此需要 line search：沿自然梯度方向指数衰减步长逐步回退，找到同时满足 `ls_surrogate > surrogate` 且 `ls_kl <= self.delta` 的最大步长（即 `candidate_params = old_params + alpha * stepsize * gradient_direction`）；若全部步长失败则恢复旧参数。

2 探索部分

请从以下五个环境中至少选择三个进行测试：**Humanoid-v5**, **Ant-v5**, **HalfCheetah-v5**, **Hopper-v5**, **Walker2d-v5** 并思考下述问题：

1. 在相同超参数设置下，三个环境的训练难度是否一致？请结合 reward 曲线进行分析。

回答：本文选择 **Humanoid-v5**、**HalfCheetah-v5** 和 **Walker2d-v5**。三者训练难度并不一致。在完整 rollout 实验末期，三者平均回报分别约为 1083、4276 和 4718。HalfCheetah 与 Walker2d 的 reward 曲线持续上升并收敛至较高水平；Humanoid 的回报提升较慢且最终值显著较低，说明其高维状态、动作空间及复杂身体协调要求提高了策略优化难度。

2. 哪个环境中 trajectory 与 rollout 的样本利用率差异最明显？可能的原因是什么？

回答：按实际有效 transition 占采集 transition 的比例计算，**HalfCheetah-v5** 中两种模式的差异最明显：rollout 的利用率为 100%，trajectory 的平均利用率仅为 62.50%，相差 37.50 个百分点。原因是 HalfCheetah 的 episode 通常运行至 1000 步上限，多个并行环境的结束时间高度同步；而 **TrajectoryRollout** 只需存满 10 条轨迹，当第二批 8 个环境同时完成时，仅有其中 2 条能够写入缓冲区，其余已采集轨迹不会参与本次策略更新。

3. 在 trajectory 模式和 rollout 模式下，advantage 与 return 的计算方式有什么不同？

回答：两种模式均采用 GAE：

$$\delta_t = r_t + \gamma(1 - d_t)V(s_{t+1}) - V(s_t), \quad A_t = \delta_t + \gamma\lambda(1 - d_t)A_{t+1},$$

并令 $R_t = A_t + V(s_t)$ 。区别在于，trajectory 模式分别对每条完整轨迹递推，并使用 **masks** 去除 padding；rollout 模式则分别对每个并行环境的固定长度片段递推，全部 transition 均参与更新且无需 mask。当前实现中，两种模式均将采样片段最后一步的下一状态价值设为 0；对自然终止的 trajectory 这是合理的，而对未终止即被截断的 rollout 会忽略末端 bootstrap，可能产生 advantage 与 return 的估计偏差。

2.1 Advantage Normalization 的影响

1. 请分别设置 `advan_norm=true` 和 `advan_norm=false` 进行实验。两组实验的平均回报曲线有何差异？

回答：本节固定使用 rollout 模式，仅改变 `advan_norm`。如图 1 所示，不同环境的结果差异很大。HalfCheetah 中，不归一化组最终测试平均回报约为 4608，高于归一化组的 4276；Humanoid 两组最终回报几乎相同，分别约为 1085 和 1083；Walker2d 中，归一化组最终达到约 4718，而不归一化组长期停留在约 355，几乎未能学会有效行走策略。

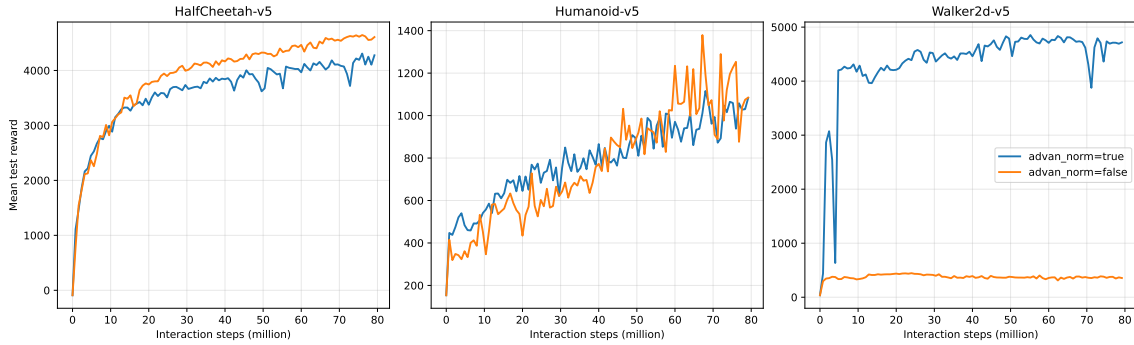


Figure 1: 三个环境中 advantage normalization 对测试平均回报曲线的影响

2. 加入 advantage normalization 后，训练过程是否更加稳定？请结合 reward 曲线、KL divergence 或 surrogate objective 的变化进行分析。

回答：总体而言，advantage normalization 提高了跨环境训练的鲁棒性，但并非在每个环境中都让 reward 曲线更平滑。最明显的例子是 Walker2d：不归一化时 surrogate objective 只有 86.6% 的记录为有限值，其有限值的最大绝对值也达到约 9.83×10^{33} ；同时平均 KL 仅为 0.00213，约 4.0% 的更新中 KL 接近 0，说明大量策略更新退化或失败。归一化后，surrogate objective 全部为有限值，平均 KL 为 0.00881，能够较稳定地利用 $\delta = 0.01$ 的信赖域并持续提高回报。Humanoid 中，归一化组最后十次测试回报的标准差约为 57.1，也明显低于不归一化组的 123.8。不过 HalfCheetah 的不归一化组同样保持了稳定训练，因此归一化的主要作用是限制不同 batch 中 advantage 尺度的剧烈变化、降低数值失稳风险，而不是保证所有曲线都更平滑。图 2 中 raw surrogate 的尺度会直接受到 advantage 缩放影响，因此更应关注是否出现非有限值以及 KL 是否退化，而不能只比较其绝对大小。

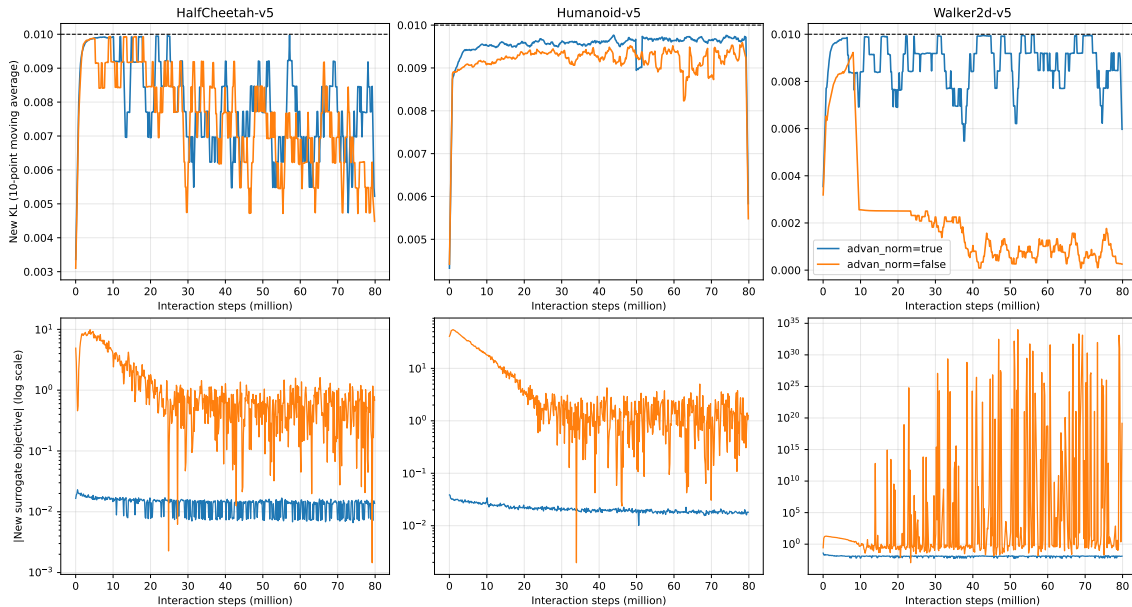


Figure 2: Advantage normalization 对 KL divergence 与 surrogate objective 的影响

3. 在你选择的测试环境中，advantage normalization 是否总是带来性能提升？如果不是，可能的原因是什么？

回答：不是。它在 Walker2d 中带来了决定性的性能提升，但在 Humanoid 中最终性能基本不变，在 HalfCheetah 中不归一化组反而取得了更高的最终回报。可能原因是 TRPO 已通过 KL 约束和 line search 限制策略更新幅度；当原始 advantage 的尺度本身较稳定时，额外归一化并非必需。减去 batch 均值还会改变有限样本下的梯度方向，并引入 batch 统计噪声，因此归一化更像是一种提高数值鲁棒性的机制，而不是必然提升最终性能的方法。

2.2 Trajectory 与 Rollout 的样本利用率比较

1. 在 rollout 模式下，每次策略更新实际使用了多少个 transition？在 trajectory 模式下，每次策略更新又实际使用了多少个有效 transition？

回答：rollout 模式配置了 8 个并行环境和 `interact_per_epoch=2000`，因此每次更新固定采集并使用

$$8 \times 2000 = 16000$$

个 transition。trajectory 模式每次收集 10 条轨迹，有效 transition 数等于 `masks.sum()`，会随 episode 长度变化。对全部 5000 次策略更新取平均后，HalfCheetah、Humanoid 和 Walker2d 每次分别实际使用约 10000、884.81 和 6224.58 个有效 transition；对应的平均采集数分别为 16000、1284.45 和 8780.30。

2. 对比 trajectory 与 rollout 两种模式，哪一种样本利用率更高？

回答：如图 3 所示，rollout 中所有固定长度片段都会参与更新，因此三个环境的利用率均为 100%。trajectory 只使用成功写入 `TrajectoryRollout` 的完整轨迹，三个环境的平均利用率分别为 62.50%、69.04% 和 70.75%。因此按照“参与更新的有效 transition 数 / 实际采集 transition 数”定义，rollout 的样本利用率更高。

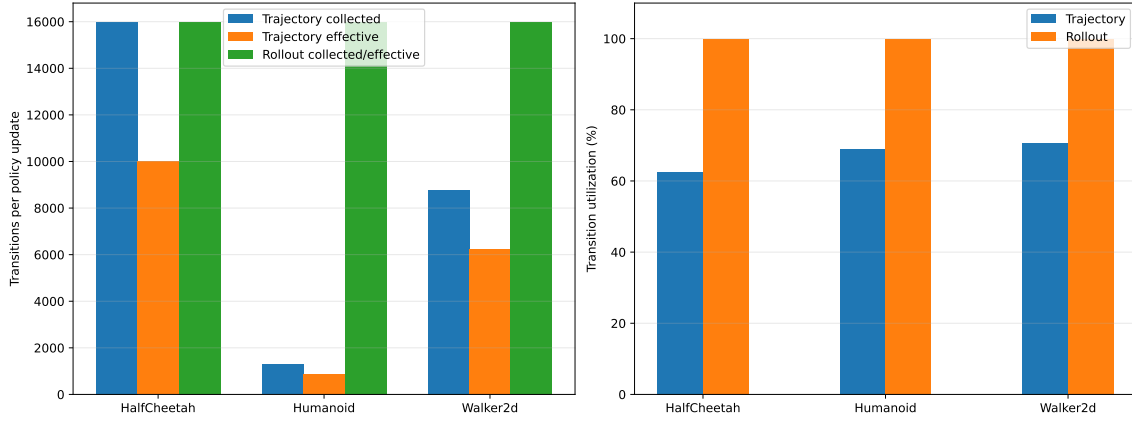


Figure 3: Trajectory 与 rollout 每次更新的 transition 数量及样本利用率

3. 样本利用率更高是否一定意味着训练效果更好？请结合实验曲线进行分析。

回答：不一定。如图 4 所示，在相同交互步数处，HalfCheetah 的 trajectory 与 rollout 回报分别约为 1727 和 4276，高利用率 rollout 明显更好；但 Humanoid 在约 6.34×10^6 个交互步时，利用率较低的 trajectory 回报约为 655，高于 rollout 的约 459；Walker2d 在约 4.33×10^7 步时，trajectory 与 rollout 也分别约为 4713 和 4657。这说明样本数量和样本质量需要同时考虑：rollout 不丢弃 transition，但固定长度截断可能引入 GAE 边界偏差；trajectory 会浪费并行采集产生的部分数据，却保留了更完整的 episode 边界。因此，更高的 transition 利用率并不保证更高的 reward。

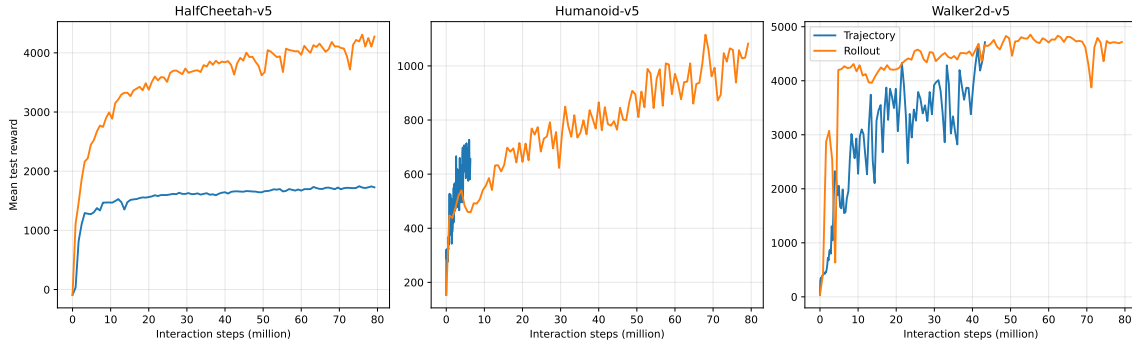


Figure 4: Trajectory 与 rollout 的测试平均回报曲线

3 总结与反思

1. 在本项目中，collect_trajectories 的实现是否会改变 TRPO 的 on-policy 特性？说出你的判断理由。

回答：不会。trajectory 与 rollout 模式均使用当前策略采集数据，并在一次策略更新后清空对应缓冲区，不会重复使用旧策略生成的样本。因此，collect_trajectories 仅改变采样停止条件和数据组织形式，不改变 TRPO 的 on-policy 特性。

2. 如果只能保留一种采样方式，你会选择 trajectory 还是 rollout？请结合样本利用率和训练性能说明理由。

回答：如果只能保留一种，我会选择 rollout。最直接的原因是它采集的 transition 都会参与本次更新，三个环境中的样本利用率都是 100%，而 trajectory 只有约 62.50% ~ 70.75%。从实验结果看，rollout 在 HalfCheetah 上明显优于 trajectory，在 Walker2d 上最终性能也很好，说明固定长度截断带来的 advantage 估计偏差不一定会成为主要问题。另外，rollout 每次更新固定使用 16000 个 transition，训练批量更容易控制，比较不同实验时也更方便。

但这个选择并不是说 trajectory 没有价值。Humanoid 和 Walker2d 在部分相同交互步数下，trajectory 的回报反而更高，说明完整轨迹确实可能带来更准确的 advantage。只是综合样本利用率、实现复杂度和三个环境的整体结果，我认为 rollout 更适合作为默认采样方式。如果继续改进，可以给 rollout 末端加入正确的 value bootstrap，尽量减小固定长度截断造成的偏差。

3. 完成实验报告的撰写后，你对 TRPO 算法是否有了新的感悟或认识？(这个问题不建议 AI 辅助生成，AI 时代最珍贵的是人类自我的想法。相信你写完报告后总会有一些自己的思考，或许是疑惑或许是感悟，完善与否不重要，写出你自己的想法就好。)

回答：我一开始对 TRPO 的理解主要停留在“用 KL 散度限制策略更新”这一句话上，感觉它只是比普通策略梯度多了一个约束。真正补完代码以后，我才发现 TRPO 的一次更新其实包含很多互相依赖的步骤：先计算 advantage，再求 surrogate objective 的梯度，用共轭梯度求自然梯度方向，最后还要通过 line search 检查 KL 和 surrogate 是否真的满足要求。中间任何一个量出现问题，最后的策略更新都有可能失败。

这次实验里让我印象比较深的是，理论上的单调改进并不等于实际训练曲线一定单调上升。理论依赖精确的期望和约束，但代码里用的是有限样本、估计出来的 advantage 和 KL 的二阶近似，所以仍然需要 line search 做最后检查。Walker2d 关闭 advantage normalization 后，surrogate objective 甚至会出现非有限值，也让我意识到一个看起来很简单的数据标准化操作，可能直接决定算法能不能正常训练。相比只看公式，这次从采样、更新到曲线分析完整跑一遍后，我对 TRPO 为什么强调“限制每次更新幅度”理解得更具体了。

4. 针对本次算法实现，你是否有什么想法和想改进的建议，你在实现过程中有没有因为项目 README.md 或者实验报告引导和介绍问题导致卡了很久，如果有，请详细说明，并给出你的建议。关于代码/实验报告问题难度你是否有感觉很难/简单？

回答：这次实现中，我卡得最久的地方不是某个公式本身，而是公式和代码中的数据形状、缓冲区状态对应不上。例如 rollout 缓冲区刚好填满时，ReplayBuffer.pos 会回到 0，如果继续把 pos 当作有效长度，就会取到空数据，最后在计算 log probability 或 reshape 时才报错。这个错误离真正原因比较远，排查起来很耗时间。后来需要根据 full 判断有效长度，缓冲区满时使用 buffer_size。另外，服务器连接中断、SwanLab 无法连接以及长时间实验失败后重新运行，也占用了不少时间。

README 目前主要介绍了环境安装和两个运行脚本，但对配置项、输出目录和常见错误解释得比较少。我建议补充一组能够快速验证代码的短实验命令，并说明 collect_traj、buffer_name、interact_per_epoch 和 advan_norm 等参数的作用；同时给出服务器无法连接 SwanLab 时使用 TensorBoard 保存日志、再复制到本地分析的备用流程。代码中的 TODO 即使已经实现仍然保留，也容易让人误以为对应部分还没有完成，最好在实现后删除或改成普通注释。

我认为这次任务的难点属于“看懂公式不算特别难，但正确实现并验证比较难”。GAE、共轭梯度和 line search 单独看都能理解，真正困难的是保证 mask、episode 边界、buffer 长度和 tensor shape 全部正确，并通过多个环境的长时间实验确认实现没有隐藏问题。如果项目能够提供少量针对 advantage 计算、满缓冲区和 trajectory mask 的单元测试，会减少很多排查时间。